Algorithmic Trading Platform — Authentication and Authorisation

Overview

This document outlines the authentication workflow for the algorithmic trading platform. The system supports standard email/password registration and sign-in, as well as third-party identity providers, JWT-based session management, and a secure, passwordless recovery mechanism using passkeys.

The passkey flow is triggered only when a user initiates the "Forgot Password" process. It is not activated during regular sign-in or as part of post-login setup. Passkey serves as the exclusive method for account recovery and future passwordless authentication in this scenario.

1. High-Level Architecture

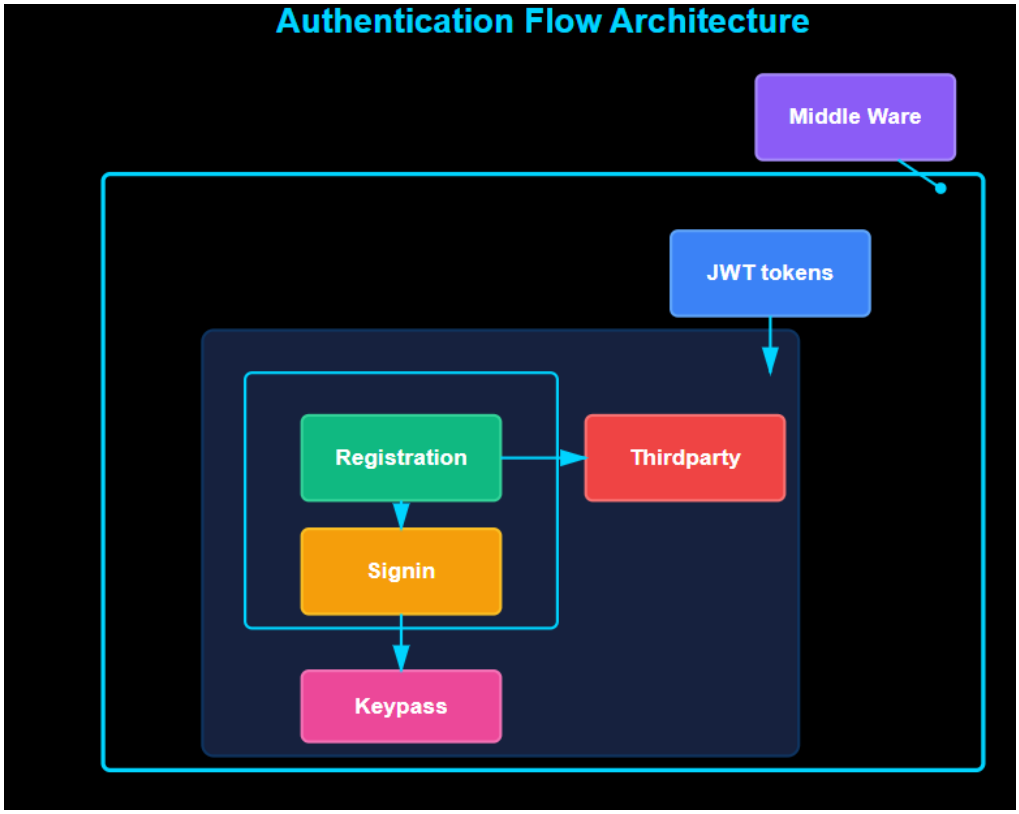
The authentication system for the algorithmic trading platform is designed to be secure, scalable, and resilient. It follows a layered architecture that enforces strict access control, supports multiple identity sources, and leverages modern passwordless standards for recovery. All user interactions flow through a protected boundary, ensuring consistent security enforcement across all entry points.

- **System boundaries**

All authentication-related operations occur within a secured perimeter (represented visually by the outer dashed box in the diagram). This boundary encapsulates:

- User-facing flows (Registration, Sign-In, Passkey Recovery)
- Token issuance and management
- Integration with external identity providers
- Middleware enforcement layer

This design ensures that no request bypasses security validation before reaching core application logic.



CORE COMPONENTS

Here are the key components visible in the diagram:

1. Registration – User account creation.
2. Sign in – User login process.
3. Key pass – Likely a password manager or secure credential handler (could be an internal service or external).
4. Third-party – Integration with external identity providers (e.g., Google, Facebook, OAuth2 services).
5. JWT tokens – JSON Web Tokens used for stateless authentication after login.
6. Middleware – Middleware layer that intercepts requests to validate tokens and enforce access control.

WORKFLOW EXPLANATION (STEP-BY-STEP)

STEP 1: USER REGISTRATION

- A new user signs up via the Registration module.
- This typically involves collecting email/password, validating inputs, and storing hashed credentials securely.
- Upon successful registration, the system may optionally:
    - Send verification email.
    - Automatically log the user in.
    - Redirect to the login page.

STEP 2: USER SIGN-IN

- Existing users authenticate through the Signin module.
- They provide credentials (email + password), which are validated against stored records.
- After successful validation, the system generates a JWT token (see Step 4)

STEP 3: KEYPASS INTEGRATION

- The Keypass component appears after Sign in — suggesting it's involved in handling or verifying passwords securely.
- Possible roles of Keypass:
    - Password hashing/validation.
    - Secure storage/retrieval of credentials.
    - Integration with hardware security modules or password managers.
- It acts as an internal security layer between Signin and token generation.

STEP 4: JWT TOKEN GENERATION & USAGE

- Once authentication succeeds (via Signin or Thirdparty), the system generates a JWT token.
- JWT contains claims like:
    - User ID
    - Expiration time
    - Roles/permissions
    - Issuer info
- This token is sent back to the client (browser/app) and stored (usually in localStorage or cookies).

STEP 5: MIDDLEWARE VALIDATION

- All subsequent API requests from the client must include the JWT token (typically in the Authorisation: Bearer <token> header).
- The MiddleWare intercepts these requests before they reach backend controllers/services.
- Middleware validates the JWT:
    - Checks signature integrity.
    - Verifies expiration.
    - Ensures issuer and audience are correct.
- If valid → request proceeds.
- If invalid/expired → returns 401 Unauthorized.

OPTIONAL: THIRD-PARTY AUTHENTICATION FLOW

The diagram shows two paths to Thirdparty:

1. From Registration: Allows users to register using social accounts (OAuth2).
2. From Signin: Allows users to log in using existing social accounts.

In both cases:

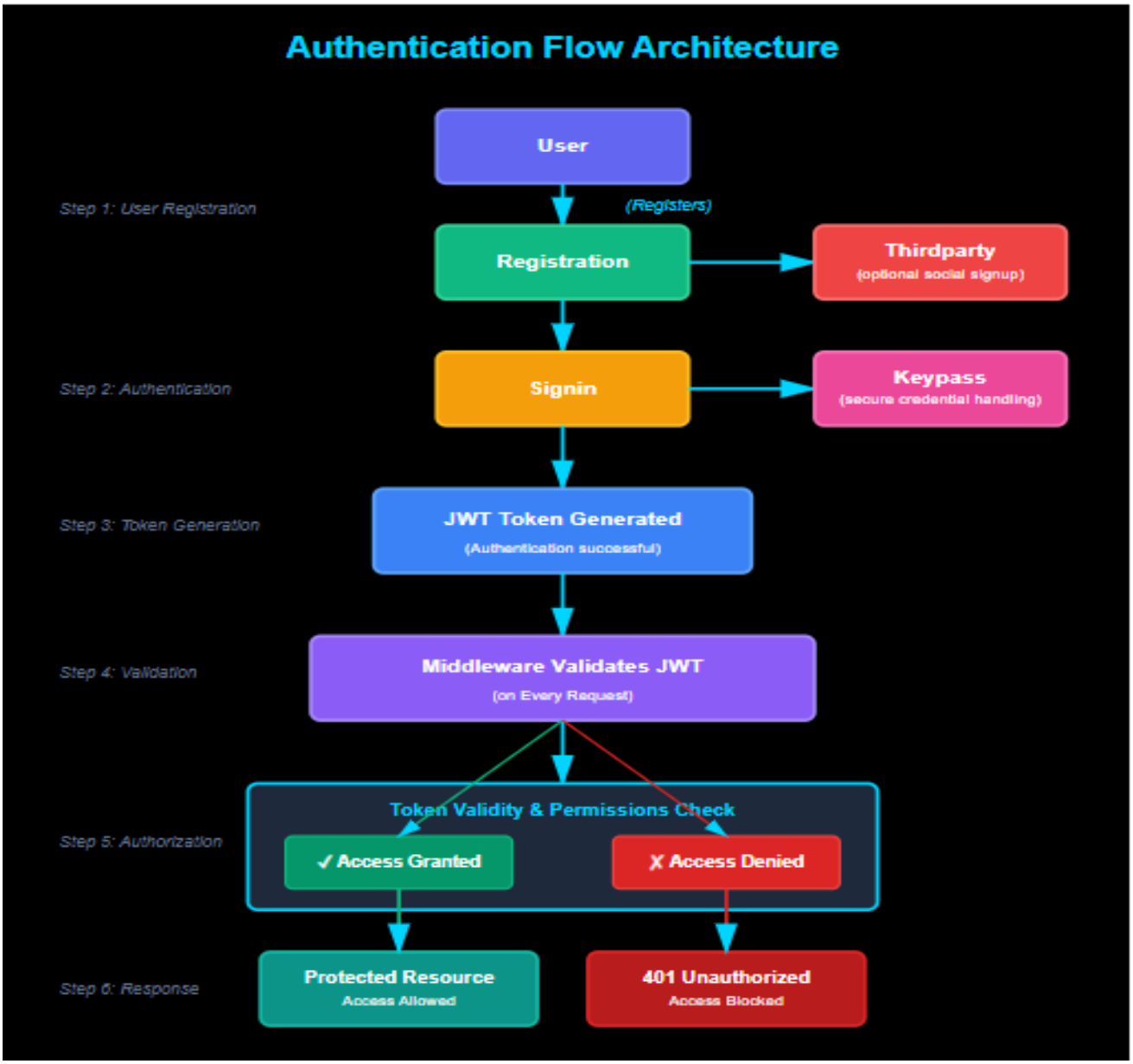
- User is redirected to third-party provider (Google, etc.).
- After consent, the provider sends back an access token or ID token.
- Your system verifies this token, creates or links a local user account if needed.
- Then generates its own JWT token for internal use.

BENEFITS OF THIS ARCHITECTURE

- Stateless: Uses JWTs instead of server-side sessions → scalable.
- Secure: Passwords handled via Keypass; tokens signed and verified.
- Flexible: Supports both native and third-party authentication.
- Modular: Middleware decouples auth logic from business logic.
- Extensible: Easy to add new identity providers or change token policies.

POTENTIAL CONSIDERATIONS / IMPROVEMENTS

- Add refresh token mechanism for long-lived sessions.
- Implement rate limiting on Signin/Registration to prevent brute force.
- Use HTTPS everywhere to protect tokens in transit.
- Consider short-lived JWTs + silent refresh for better security.


Authentication Flow Architecture
User
Step 1: User Registration
(Registers)
Registration
Thirdparty
(optional social signup)
Step 2: Authentication
Signin
Keypass
(secure credential handling)
Step 3: Token Generation
JWT Token Generated
(Authentication successful)
Step 4: Validation
Middleware Validates JWT
(on Every Request)
Step 5: Authorization
Token Validity & Permissions Check
✓ Access Granted
✗ Access Denied
Step 6: Response
Protected Resource
Access Allowed
401 Unauthorized
Access Blocked

Authorisation Framework for Algorithmic Trading System

Introduction

Authorisation in an algorithmic trading system defines what authenticated users or services are permitted to do within the platform. Given the high financial stakes, regulatory requirements, and operational risks, a robust authorisation framework is essential to enforce least privilege, segregation of duties, and real-time risk controls.

This document outlines the key authorisation techniques that can be integrated into the system, building upon the existing authentication flow (e.g., JWT-based login with third-party support).

2. Core Authorisation Techniques

2.1 Role-Based Access Control (RBAC)

Description:
Access rights are granted based on predefined user roles.

Role	Permissions
Trader	Create/modify strategies, view own positions, execute trades (within limits)
Risk Manager	View all positions, override risk limits, and approve high-value trades.
Compliance Officer	Audit logs, monitor trades, and enforce regulatory rules.
Admin	Manage users, configure system settings, revoke sessions.
Viewer	Read-only access to dashboards and reports

Implementation:

- Roles are embedded as claims in the JWT token during authentication.
- Middleware validates the role claim against endpoint requirements.

Use Case:
Prevents junior traders from modifying risk parameters or accessing other users' strategies.

2.2 Attribute-Based Access Control (ABAC)

Description:
Dynamic authorisation decisions based on attributes of the user, resource, action, and environment.

Example Attributes:

- User department (quant_team, execution_desk)
- Time of day (market_hours = true)
- Portfolio risk score (drawdown < 5%)
- IP address (trusted_network = true)

Policy Engine:
Integrated with Open Policy Agent (OPA) or a custom policy evaluator.

Sample Policy:

"Allow trade execution only if:

- User role is trader,
- Current time is within market hours,
- Strategy risk score ≤ 0.6."

Use Case:
Automatically blocks trades during after-hours or when portfolio risk exceeds thresholds.

2.3 OAuth 2.0 Scopes (Fine-Grained Permissions)

Description:
Granular permissions expressed as OAuth scopes are especially useful for API access and third-party integrations.

Common Scopes:

- read: positions
- write: strategies
- execute: trades
- admin: users

Implementation:

- Scopes are issued in the JWT during login or token exchange.
- Middleware validates that the requested operation matches at least one granted scope.

Use Case:
A third-party analytics dashboard is granted only read: positions, preventing trade manipulation.

2.3 Segregation of Duties (SoD)

Description:
Ensures no single user can perform conflicting operations that could lead to fraud or error.

Enforced Rules:

- A user who creates a trading strategy cannot approve its live execution.
- Trade initiation and trade cancellation must involve different personnel for large orders.

Implementation:

- Workflow engine tracks action ownership.

- Authorisation middleware checks for SoD violations before allowing state transitions.

Use Case:
Meets regulatory requirements (e.g., MiFID II, SOX) by enforcing dual controls.

3. Advanced Authorisation Controls

3.1 Step-Up Authentication (Contextual MFA)

Description:
Requires additional verification (e.g., OTP, biometric) for high-risk actions.

Triggered For:

- Executing trades above a threshold (e.g., > $100,000)
- Deleting active strategies
- Accessing admin or risk override panels

Implementation:

- Initial JWT grants baseline access.
- Sensitive endpoints trigger MFA challenge.
- Upon successful verification, a short-lived elevated token is issued.

Use Case:
Mitigates risk of account compromise leading to unauthorised large trades.

3.2 Session & Token Management

Features:

- Short-lived access tokens (e.g., 15–30 minutes)
- Token revocation via admin panel or automated alerts
- Active session tracking (stored in Redis with TTL)

Security Benefit:
Limits exposure in case of token leakage and supports immediate deactivation.

3.3 Session & Token Management

Description:
Restrict system access to trusted networks or geographic regions.

Features:

- Short-lived access tokens (e.g., 15–30 minutes)
- Token revocation via admin panel or automated alerts
- Active session tracking (stored in Redis with TTL)

Security Benefit:
Limits exposure in case of token leakage and supports immediate deactivation.

3.4 Rate Limiting & Quota Enforcement

Description:
Control the frequency and volume of user actions.

Examples:

- Max 50 trade executions per hour per user
- Max 5 strategy deployments per day

Implementation:
Use Redis counters with sliding windows. Reject requests exceeding quotas.

Use Case:
Prevents runaway algorithms or abuse of the trading API.

Conclusion

By implementing a layered authorisation framework combining RBAC, ABAC, OAuth scopes, SoD, and contextual controls, the algorithmic trading system achieves:

- Strong security posture
- Regulatory compliance
- Operational resilience
- Auditability and accountability

This design ensures that only authorised users can perform appropriate actions under safe and monitored conditions—a critical requirement for any financial trading platform.